SIH 2026 PS 26262: Work Split for a Team of Three

Based on	SIH26262_Final_Report (frozen research baseline, 24 Oct 2026)
Team	Person 1 (P1), Person 2 (P2), Person 3 (P3)
Split	P1 Approval workflow + EVC · P2 Documents/OCR + Compliance + Deadlines · P3 Frontend + Dashboards + Security + Audit
Purpose	Every part of the architecture has exactly one owner, and every point where two people's work meets is defined in advance
Repository	`approval-workflow-ocr-compliance` (private, GitHub)
Branches	`main` (always working) + `person-1`, `person-2`, `person-3`; integration into `main` every 2–3 days
Status (24 Oct 2026)	Day 0 setup and the Week 1 slice are built and pushed; 31 tests pass (section 13)

Section references such as "§19" point to the final report.

Contents

1. How the split works
2. The three rules that prevent conflicts
3. Person 1: Approval workflow and EVC
4. Person 2: Documents, OCR, compliance and deadlines
5. Person 3: Frontend, dashboards, security and audit
6. Interfaces between people (the contracts)
7. Day 0: what all three do together
8. Conflict register: overlap points and who wins
9. Coverage check: nothing is left unowned
10. Week-by-week plan
11. Effort balance and rebalancing levers
12. Demo: who presents what
13. What is already built (Week 1 slice)
14. Branches and integration

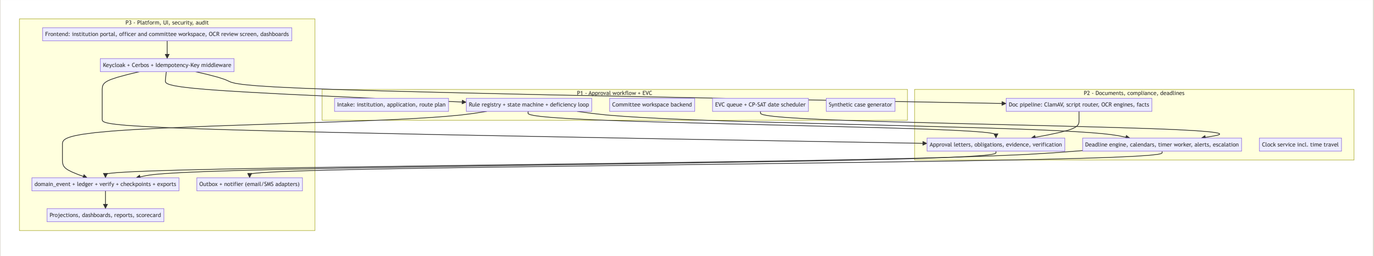
---

1. How the split works

The system is one FastAPI + PostgreSQL modular monolith (§17). The split follows module boundaries, not technical layers. Each person owns:

- a set of **backend folders**;
- a **PostgreSQL schema**, which no one else writes to;
- the **API endpoints** for their domain;
- the **events** their module writes to the ledger;
- the **tests** proving their acceptance criteria.

The one exception is the frontend. **P3 builds every screen**, and P1 and P2 supply the API and a wireframe or acceptance test for each screen in their domain. One person owning the UI keeps the look, routing and auth consistent and stops three people editing the same React app.


P3 - Platform, UI, security, audit
Frontend: institution portal, officer and committee workspace, OCR review screen, dashboards
Keycloak + Cerbos + Idempotency-Key middleware
domain_event + ledger + verify + checkpoints + exports
Outbox + notifier (email/SMS adapters)
Projections, dashboards, reports, scorecard
P1 - Approval workflow + EVC
Intake: institution, application, route plan
Rule registry + state machine + deficiency loop
Committee workspace backend
EVC queue + CP-SAT date scheduler
Synthetic case generator
P2 - Documents, compliance, deadlines
Doc pipeline: ClamAV, script router, OCR engines, facts
Approval letters, obligations, evidence, verification
Deadline engine, calendars, timer worker, alerts, escalation
Clock service incl. time travel

---

2. The three rules that prevent conflicts

Rule 1: one writer per table. Each table belongs to exactly one person's PostgreSQL schema (`wf`, `evc`, `doc`, `cmp`, `dl`, `plat`, `an`). Code only *writes* its own schema. To change someone else's data, call their port function (section 6), never an UPDATE on their table. Reads across schemas go through read-only views that the owner publishes.

Rule 2: one owner per folder. The folder-to-person table is in `docs/OWNERSHIP.md`, and it uses only "Person 1/2/3". `CODEOWNERS` lists all three GitHub usernames for every path, so GitHub asks the team for review without naming who owns which part. Folder ownership is therefore a **written team rule**: change someone else's folder only with their agreement. `contracts/` needs approval from all three. CODEOWNERS accepts any one owner's approval, so a separate CI check (`contracts-approval`) enforces the three-way approval.

```
approval-workflow-ocr-compliance/
  contracts/                    ALL THREE (CI blocks merge until both other members approve)
    ports.py                    Python Protocols modules call each other through
    types.py, enums.py          shared value types and vocabularies (roles, stages, states, prefixes)
    errors.py                   errors any module may raise (mapped to HTTP codes)
    fakes.py                    in-memory stand-in for every port
    events/<prefix>/<name>.v<N>.json   JSON Schema per event; no free-text fields allowed
    openapi.json                snapshot of the real API; CI fails if it drifts
  backend/app/
    platform/                   P3  unit of work, demo auth + policy, ledger + verify, notes,
                                    personal data, outbox (next: Keycloak, Cerbos, notifier, idempotency)
    intake/                     P1  institutions, applications, submission
    workflow/                   P1  ruleset registry (next: state machine, deficiency loop, committees)
    evc/                        P1  queue, expert pool, COI, selector stand-in, CP-SAT scheduler
    documents/                  P2  upload, ClamAV, router, OCR engines, extractor, facts
    compliance/                 P2  approval letters, obligations, evidence, verification
    deadlines/                  P2  rules, due-date math, engine, clock (next: calendars, timer, alerts)
    analytics/                  P3  projections, dashboards, reports, scorecard
    main.py                     P3  discovers each module's models.py, wiring.py and api.py
  backend/migrations/versions/<module>/   each owner: one Alembic chain per module (Rule 3)
  rulesets/aph-2024-27/r1/routes.yaml       P1
  rulesets/aph-2024-27/r1/deadlines.yaml    P2
  rulesets/aph-2024-27/r3-DEMO/overlay.yaml P1 (deadline keys need P2 approval)
  calendars/                    P2
  generator/cases/              P1  (seed_demo.py today; scale generator next)
  generator/documents/          P2  (also the OCR evaluation set)
  frontend/                     P3
  docker-compose.yml, backend/Dockerfile, .github/, scripts/   P3
  tests/e2e/stepNN_*_test.py  owner of each demo step (section 12): one file each
  docs/                         ARCHITECTURE, OWNERSHIP, TEAM_WORKFLOW
```

Rule 3: no shared migration head. Each module keeps its own Alembic chain in `backend/migrations/versions/<module>/` with its own `branch_labels`, so three people never edit the same revision file. Upgrade with `alembic -c backend/alembic.ini upgrade heads` (plural). `scripts/check_migration_heads.py` runs in CI and fails if any module accidentally grows a second head. Cross-schema foreign keys are allowed only to the stable `id` of a table owned by someone else, and that owner promises never to rename or drop it.

Rule 4: one command, one transaction. A workflow transition, the deadlines it opens, its ledger events and any outbox rows commit atomically in PostgreSQL (`unit_of_work`). There is no asynchronous event processing for mandatory work such as creating a deadline. Only notification **delivery** happens later: Person 3's notifier reads the outbox after commit.

Supporting habits:

- **Event names carry the module prefix:** `wf.*`, `evc.*`, `doc.*`, `cmp.*`, `dl.*`, `plat.*`. Two people therefore cannot invent the same event name.
- **Stubs first.** Every port has a fake in `contracts/fakes.py`, so nobody waits on anyone.
- **Person branches, frequent integration.** Each person works on their own `person-N` branch. Branches are integrated into `main` every 2–3 days (section 14).
- **Lightweight local setup.** `docker compose up` runs only Postgres, the backend and the frontend. Heavier services (Keycloak, Cerbos, ClamAV, MinIO) come later under an optional profile.
- **A 15-minute daily sync** covers only what is blocked and which contract needs to change.

---

3. Person 1: Approval workflow and EVC

Mission. An application enters, follows the correct Table 2.3 route for its pinned ruleset version, moves only through valid transitions, and reaches EVC with conflict-free dates.

3.1 Owns

Area	Details	Report
Intake	`institution`, `application`, `submission`; category checklist (Annexure-2); DSC flag; "can't submit with mandatory items missing"	§17.1
Route plan	On submit, return the route (e.g. Scrutiny → EVC → SHC as applicable) with clause citation	§4.2, §18
Rule registry	Loads, versions and pins rulesets (`aph-2024-27@r1`); static validation; approval flow for new rulesets; `r3-DEMO` overlay; rule-change impact replay (Tier 2)	§18
`routes.yaml`	All 21 Table 2.3 routes plus NEW_INSTITUTION (Chapter I); SHC "as applicable" exactly where EVC = Yes	§4.2
State machine	`route_def`, `ruleset`, `transition`, `stage_visit`; guards; reverts with reasons; 100% of invalid transitions rejected	§18
Deficiency loop	`deficiency`, `deficiency_item`; Re-Scrutiny scope = open items only; items traceable to responses	§4.4
Committee workspace (backend)	`committee`, `task`, `observation`; append-only report versions attributed to members	§17.1

Area	Details	Report
EVC	`evc_request`, `expert`, `availability`, `coi`, `slot`, `assignment`; labelled stand-in "authorised selector" for panels (APH 1.8(a): automated, **not random**); CP-SAT chooses **dates only**; EDF baseline; "why this slot" explanations; propose/publish	§23
Letter issuance trigger	On LoA/EoA/LoD issue, call P2's `register_letter`	§6.2
Mock adapters	AICTE portal events, NSWS (labelled SIMULATED); inbound dedup	§17
Synthetic case generator	~300 institutions, 9 ROs, ~600 applications in the Table 2.3 mix, ≥5 seeds; planted workflow events (deficiency loops, late responses, EVC delays); writes the answer key	§28.2
Tier 2 extras	BPMN view of route versions; cited case summaries (local LLM, optional)	§28.1
Spike	Operaton 2-day decision spike (alternative workflow core)	§18

3.2 APIs owned

- `POST /applications` · `POST /applications/{id}/submit`
- `POST /cases/{id}/actions` (all transitions; guarded and idempotent through P3's middleware)
- `GET /evc/queue` · `POST /evc/schedule:propose` · `POST /evc/schedule:publish`
- `POST /admin/rulesets` · `POST /admin/rulesets/{id}/validate` · `POST /admin/rulesets/{id}/approve`

3.3 Calls other people's ports

Calls	Owner	When
`deadlines.open / close / pause_internal / resume_internal`	P2	Every transition whose YAML has `timers:` (e.g. record_deficiency pauses AICTE_PROCESSING and starts INSTITUTION_RESPONSE)
`compliance.register_letter`	P2	LoA/EoA/LoD/corrigendum issued
`ledger.append`, `notes.create`, `authz.check`, `uow`	P3	Every command

3.4 Acceptance tests (P1 proves these)

- Each of the 21 routes plus NEW_INSTITUTION produces the stage list in §4.2. This is a table-driven test straight from the YAML.
- 100% of invalid transitions are rejected with a reason code.
- A case pinned to r1 stays on r1 after r3-DEMO is loaded, and the rule-version switch shows the difference (demo step 10).
- Re-Scrutiny shows only open items.
- The scheduler reports 0 hard-constraint violations (COI, availability, no expert double-booked) and beats EDF on the same queue.

3.5 Open questions P1 chases (§34)

- Q1: official APH URL plus amendments. The Dec 2025 corrigendum stays r3-DEMO only until it is confirmed.
- Q2: academic calendar EVC and LoA dates.
- Q3: Annexure-7 flow chart and the full Annexure-2 lists per category.
- Q5: number of Regional Offices.

No APH source was found for EVC duration or for the deficiency-response window. Both remain configurable demo assumptions in YAML, labelled as such.

---

4. Person 2: Documents, OCR, compliance and deadlines

Mission. Every legal or internal deadline is computed correctly and explainably, every obligation has a source, an owner and a due date, and no deadline is ever created from an unconfirmed OCR value.

4.1 Owns

Area	Details	Report
Document lifecycle	`document`, `document_version` (content-addressed in the private object store); ClamAV scan; upload API	§20
OCR pipeline	Docling/pypdfium2 text layer → script router → **Tesseract (Latin)** / **Indic ONNX model (Devanagari, Kannada...)** → digit normalisation → date grammar → `fact` (state PROPOSED)	§20, §21
Indic OCR app integration	Stage 0 fixes (HTTP 500 on corrupt files, EXIF rotation, digit lookalikes, confidence not calibrated); **model ownership and licence check, which gates submission**	§21.6, §22.2
Handwriting	Stage 1 detection + deskew; **Stage 2: detect handwriting → route to human with the crop, never guess**; Stages 3–4 are roadmap only	§22
Commit-before-reveal (backend)	Officer types the value first, then sees the suggestion; state goes CONFIRMED / CORRECTED / REJECTED	§20.2
OCR evaluation set	Synthetic lease deeds per script and degradation level; seeded reviewer errors (2–5%); both metrics from §22.4	§22.4
Approval letters	`approval_letter` (issue_date, issue_date_source, issue_date_fact_id, lists_deficiencies)	§27
Obligations	`obligation` (kind UNDERTAKING / AFFIDAVIT / LETTER_CONDITION / DEPOSIT / DOCUMENT_VALIDITY / RECURRING_FILING; provenance and confirmation fields), `evidence`, `verification` (human only, reason on reject); ACK ≠ MET	§7, §27
`deadlines.yaml`	EOA_DEFICIENCY_6M (applies only when the letter lists deficiencies, month-end clamp), SAC 7-day appeal (2.24), deposit 15 days + 10%/20% ladder (1.12), LoA validity 2 years (1.10(b)), lease stop-AY (2.1(e), inclusive end date), internal processing clocks	§19
Deadline engine	`deadline` (LEGAL vs INTERNAL clocks, clock owner, due_version), `calendar_version`, working-day calendars; extensions; `explain`	§19
Timer worker	`SKIP LOCKED` polling, catch-up after restart	§19
Alerts and escalation	`alert` (UNIQUE deadline + level + due_version, so duplicates are impossible), `ack`, `ladder` (T-3 reminder → RO → Bureau; vacant post handling)	§19
Clock service	`clock.now()` for the whole system; `POST /sim/clock` time travel (demo only, disabled in production)	§28.3
Document generator	DEMO-stamped documents: valid, expired, near-expiry, unclear, corrected, duplicate	§28.2
Mock adapter	DigiLocker / API Setu (labelled SIMULATED)	§17
Tier 2 extras	Rule-based early warning (deadline risk)	§28.1

4.2 APIs owned

- `POST /documents` · `GET /documents/{id}/facts` · `POST /facts/{id}/confirm`
- `GET /obligations` · `POST /obligations/{id}/evidence` · `POST /evidence/{id}/verify`
- `GET /deadlines?owner=&status=` · `GET /deadlines/{id}/explain` · `POST /deadlines/{id}/extensions`

- `GET /alerts` · `POST /alerts/{id}/ack`
- `POST /sim/clock`

4.3 Ports P2 provides (P1 and P3 call these)

`deadlines.*`, `compliance.register_letter`, `clock.now` (section 6).

4.4 Calls other people's ports

Calls	Owner	When
`outbox.enqueue(AlertMessage)`	P3	Each new alert row; P3 delivers it
`ledger.append`, `notes.create`, `personal.store`, `authz.check`, `uow`	P3	Every command
`rulesets.get(rev).deadline_rule(id)`	P1	Reading the pinned deadline rule; P1's registry loads P2's `deadlines.yaml` through a validator function that P2 supplies

4.5 Acceptance tests (P2 proves these)

- Golden date tests for every rule in `deadlines.yaml`, including month-end clamp, working days across holidays, and the lease off-by-one case (lease ending 30 Jun 2031 with D=4 → stop AY 2027-28).
- LEGAL clocks are never paused by a workflow action; INTERNAL clocks pause and resume.
- **Unconfirmed-fact blocking = 0**: no deadline can reference a PROPOSED fact (enforced by constraint and tested).
- End-to-end false-expiry alerts are measured with seeded reviewer errors and reported with a rate, without claiming 0.
- 0 duplicate alerts are generated and 0 are missed over the scale set; the timer catches up after a forced restart.
- Corrupt files fail cleanly (no 500), EXIF-rotated photos read correctly, and `12-03-2024` normalises from Devanagari lookalikes.

4.6 Open questions P2 chases (§34)

- Q4: which undertakings AICTE wants tracked.
- Q6: whether "within 7 days" means calendar or working days, inclusive or exclusive. Configurable until answered.
- Q7: OCR model ownership, training data and licence. **This gates submission.**
- Q8: handwritten dataset licences, only if Stage 3 is attempted.

---

5. Person 3: Frontend, dashboards, security and audit

Mission. Everyone sees only what their role allows, every action is recorded tamper-evidently and can be verified, and every number on a dashboard can be reproduced from the ledger.

5.1 Owns

Area	Details	Report
Frontend (all screens)	Institution portal; officer and committee workspace; **OCR review screen** (commit-before-reveal UX to P2's acceptance test); EVC schedule view; obligation and evidence screens; alert inbox with ack; "Why is this due?" panel; time-travel control (demo flag); ledger-verify screen	§17, §29
Identity and permissions	Keycloak realm (roles, regions); Cerbos policies (role, region, stage, assignment; object-level checks); cross-role matrix test	§25, §26
API platform	Unit-of-work (one Postgres transaction per command); `Idempotency-Key` middleware; error format; OpenAPI publishing	§17
Event store and ledger	`domain_event` (JSON-Schema-validated payloads, no free text), `ledger_entry` (event_hash, prev_hash, entry_hash, signature), `case_note` (note_hash), `personal_data` (erasable), signed **independent daily checkpoints**, `GET /ledger/verify`	§27
Replay	Harness that rebuilds every projection from `domain_event` and diffs it against live tables; each owner supplies projectors for their events	§27
Outbox and notifier	`outbox`, `delivery`; dispatcher with backoff, dead-letter state, and an alert when delivery itself fails; Apprise/SMTP/SMS adapters (SMS mocked)	§17, §19
Dashboards and reports	Pendency by RO and stage, workload, obligation status; `GET /dashboards/pendency`, `GET /reports/{type}`; snapshot hash so a report is reproducible	§17.1
Exports	`POST /exports`; BSA 2023 s.63-ready bundle (Tier 2); OCEL export	§25, §28
Scorecard	Reads the answer key (P1 and P2 generators) and outputs: missed alerts, duplicates, escalation correctness, OCR date accuracy per script and degradation level (metric code from P2), ledger tamper detection	§30
Security and law	Threat-model tests (§26); CERT-In 180-day logs in India now, DPDP 1-year from ~13 May 2027; GIGW host-in-India note; accessibility pass; Hindi UI (Tier 3)	§25, §26
Infrastructure	docker-compose (Postgres, Keycloak, Cerbos, ClamAV, MinIO, app, workers), CI, seed scripts, reproducible deployment	§24

5.2 APIs owned

`GET /cases/{id}/timeline` · `GET /dashboards/pendency?region=` · `GET /reports/{type}` · `GET /ledger/verify` · `POST /exports`

5.3 Ports P3 provides (P1 and P2 call these)

`uow`, `ledger.append`, `notes.create`, `personal.store`, `outbox.enqueue`, `authz.check` (section 6).

5.4 Acceptance tests (P3 proves these)

- The cross-role matrix passes: an institution cannot see another institution, and an RO cannot act outside its region or stage.
- Tamper is detected 100%. Editing one row makes `/ledger/verify` report the exact entry number, relative to the last independent checkpoint.
- Replay diff = 0 on the scale set.
- Erasing a `personal_data` row or redacting a `case_note` keeps the chain verifiable.
- A notifier outage gives late-but-delivered alerts with no duplicates *sent* beyond logged retries.
- The scorecard runs on all ≥5 seeds from one command.

5.5 Open questions P3 chases (§34)

- Q9 and Q10: literature identifiers and the public dashboard/FAQ URLs, needed for the bibliography and slides.
- The AICTE records-retention schedule, listed as unknown in §27.
- Who receives the independent checkpoints. This is a team decision for the demo.

---

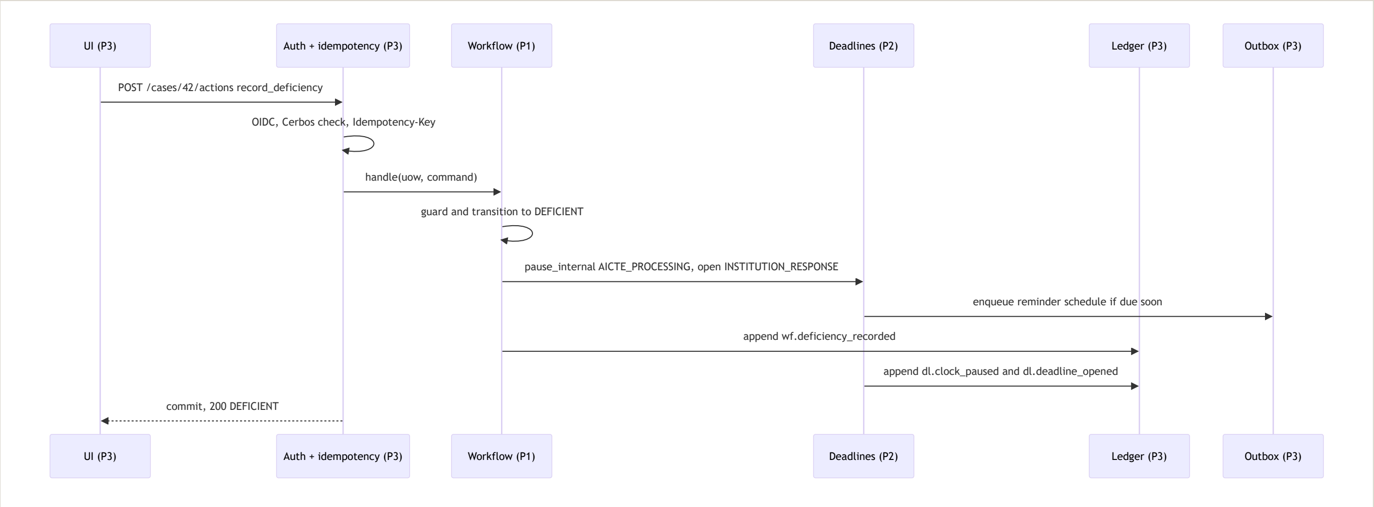
6. Interfaces between people (the contracts)

These are the signatures now in `contracts/ports.py`. A port is a Python call inside the same process and the same database transaction, not HTTP. This keeps "transition + deadlines + ledger" atomic. Each module registers its implementations in its own `wiring.py`, and callers look them up with `port(PortType)`.

```
# contracts/ports.py  (all three approve changes)

class UnitOfWork(Protocol):                     # P3 - one transaction per command
    session: Session
    principal: Principal

# ---- provided by P3 (platform) ----
class LedgerPort(Protocol):
    def append(self, uow, event: EventIn) -> int: ...           # returns seq; schema-checked, hash-chaine
    def verify(self, session) -> VerifyResult: ...
class NotesPort(Protocol):
    def create(self, uow, body: str) -> tuple[UUID, str]: ...   # (note_ref, note_hash)
class PersonalDataPort(Protocol):
    def store(self, uow, data: dict, purpose: str) -> UUID: ...
class OutboxPort(Protocol):
    def enqueue(self, uow, msg: OutboundMessage) -> None: ...
class AuthzPort(Protocol):
    def check(self, principal, action: str, resource: ResourceRef) -> None: ...

# ---- provided by P1 (workflow / intake) ----
class RulesetPort(Protocol):
    def default_key(self) -> str: ...                           # "aph-2024-27@r1"
    def get(self, key: str) -> RuleSet: ...                     # routes + parsed deadline rules
    def route_for(self, category: str, key: str) -> Route: ...
class CasePort(Protocol):
    def lookup(self, session, application_id: UUID) -> CaseRef: ...   # institution + region, for access
```
```
# ---- provided by P2 (deadlines / compliance) ----
class DeadlineRuleParserPort(Protocol):
    def parse(self, raw: dict) -> dict: ...                     # P1's registry loads deadlines.yaml throu
class DeadlinePort(Protocol):
    def open(self, uow, subject: SubjectRef, rule_id: str, anchor: Anchor, ruleset_key: str) -> UUID: ...
    def close(self, uow, subject: SubjectRef, rule_id: str, outcome: str) -> None: ...
    def pause_internal(self, uow, subject: SubjectRef, rule_id: str) -> None: ...   # LEGAL clocks refuse
    def resume_internal(self, uow, subject: SubjectRef, rule_id: str) -> None: ...
class CompliancePort(Protocol):
    def register_letter(self, uow, letter: LetterIn) -> UUID: ...
class ClockPort(Protocol):
    def now(self) -> datetime: ...                              # the ONLY source of time (time travel)
```

Shared value types (`contracts/types.py`): `Principal`, `SubjectRef(type, id)`, `ResourceRef`, `CaseRef`, `Anchor(at, event_seq)`, `EventIn(aggregate_type, aggregate_id, event_type, payload, event_version, reason_code, ruleset_rev, note_ref, note_hash, personal_ref)`, `VerifyResult`, `OutboundMessage(alert_id, channel,` `template, recipient_ref)`, `LetterIn(...)`, `RouteStage`, `Route`, `RuleSet`. Errors (`contracts/errors.py`): `ValidationFailed` 422, `NotFound` 404, `Forbidden` 403, `Conflict` 409.

6.1 One command crossing all three people


UI (P3)
Auth + idempotency (P3)
Workflow (P1)
Deadlines (P2)
Ledger (P3)
Outbox (P3)
POST /cases/42/actions record_deficiency
OIDC, Cerbos check, Idempotency-Key
handle(uow, command)
guard and transition to DEFICIENT
pause_internal AICTE_PROCESSING, open INSTITUTION_RESPONSE
enqueue reminder schedule if due soon
append wf.deficiency_recorded
append dl.clock_paused and dl.deadline_opened
commit, 200 DEFICIENT

If any step fails, the whole transaction rolls back. No half-done state exists, so there is nothing for two people to argue about.

6.2 EoA with deficiencies (P1 → P2 → P3)

1. P1: transition `issue_eoa` calls `compliance.register_letter(letter_type=EOA, lists_deficiencies=true, issue_date=portal date)`.
2. P2: creates the letter row. The officer confirms the conditions, which creates obligations with `state_origin=OFFICER_CONFIRMED`. `deadlines.open(rule EOA_DEFICIENCY_6M, anchor=letter.issue_date)` runs.
3. P2: at the T-30 / T-7 / T-1 reminder levels (§4.4) the timer creates `alert` rows and calls `outbox.enqueue`.
4. P3: the notifier delivers, retries and logs `delivery` rows. The UI shows the alert. The ack goes to P2's endpoint, which stops escalation.

6.3 Lease with OCR (P2 → P3 → P2)

1. P2: upload → ClamAV → router → OCR → `fact` PROPOSED (or a handwriting flag).
2. P3: review screen. The officer types the date, then the suggestion is revealed, then the officer confirms or corrects (calls P2's `POST /facts/{id}/confirm`).
3. P2: the confirmed fact opens the lease deadline (2.1(e) stop-AY) and the 18-month reminder.

---

7. Day 0: what all three do together

Status on 24 Oct 2026: items 1–5, 7 and 8 are **done** and pushed. Items 6 and 9 are still open and should be done together at the first team meeting.

#	Deliverable	Lead	Status
1	Repo, CODEOWNERS, CI, `docker compose up`	P3	Done

#	Deliverable	Lead	Status
2	`ports.py` + value types (section 6)	All	Done; **all three must read and agree before building on it**
3	`enums.py`: roles, stages, states, event prefixes	All	Done (reason codes added per module as needed)
4	Event schemas	Each for own module	Done for Week 1 (`wf.application_created`, `wf.application_submitted`, `dl.deadline_opened`); add yours as you build
5	API contract	Each for own endpoints	Done as `contracts/openapi.json`, generated from the real API; CI fails if it drifts
6	Answer-key schema	P1 + P3 (P2 adds OCR fields)	**Open**
7	Ruleset YAML format	P1 + P2	Done: r1 routes (all 22) and deadlines validate
8	Fake implementation of every port	Each for own ports	Done (`contracts/fakes.py`, checked by CI)
9	Wireframes for each screen	P1/P2 sketch, P3 approves	**Open**

Rule for changing a contract later: 1. Edit `contracts/` and update `contracts/fakes.py` in the same branch. 2. If an API changed, run `python scripts/check_contracts.py --write` to refresh `contracts/openapi.json`. 3. Explain the change at integration. Both other members approve; CI enforces it.

---

8. Conflict register: overlap points and who wins

These are the places where two people could both think something is theirs. Each has one owner.

Overlap point	Owner	The other person's role
Rule registry vs `deadlines.yaml`	P1 owns the registry, loader and versioning	P2 owns the deadline file contents and its validator function
`r3-DEMO` overlay	P1	P2 approves if it touches deadline keys
Approval letter: issuing it vs storing it	P1 triggers the issue	P2 owns the `approval_letter` row and everything that follows
Alert generation vs delivery	P2 creates `alert` rows, escalation and acks	P3 owns `outbox`, `delivery` and channel adapters
Ack	P2 (endpoint + logic stops escalation)	P3 builds the button
OCR review screen	P3 builds the UI	P2 owns the backend, the rules (type first, then reveal) and the acceptance test
EVC overdue escalation	P2 (it is a deadline)	P1 opens the EVC clock
Pendency numbers	P3 computes the projection	P1 owns `stage_visit` events; P2 owns INTERNAL clocks
"Why is this due?"	P2 (`/deadlines/{id}/explain`)	P3 renders it
Time	P2 `ClockPort` is the only clock	Everyone calls `clock.now()`; `datetime.now()` is banned (CI grep check)
Time-travel UI	P3	Calls P2's `/sim/clock`
Free-text reasons	P3 `case_note`	P1/P2 call `notes.create`, never put text in payloads

Overlap point	Owner	The other person's role
Personal data (names, phones)	P3 `personal_data`	P1/P2 store a reference only
Projectors for replay	Each owner for their own events	P3 owns the replay harness and diff
Synthetic data	P1 cases + EVC demand; P2 documents + OCR set	P3 consumes the answer key in the scorecard
OCR metrics in the scorecard	P2 writes the metric functions	P3 calls them and displays them
Cerbos policies	P3 writes the policy files	P1/P2 list actions and resource attributes per endpoint
Idempotency	P3 middleware	P1/P2 make handlers deterministic
Inbound adapter dedup	Adapter owner (P1 portal/NSWS, P2 DigiLocker)	Use P3's idempotency table
Institution master data	P1	P3 uses `institution_id` for scoping
User/role master data	P3	P1 references `expert.user_id`

---

9. Coverage check: nothing is left unowned

9.1 Every table has one owner

Schema	Owner	Tables
`wf`	P1	institution, application, submission, ruleset, route_def, transition, stage_visit, deficiency, deficiency_item, committee, task, observation, inbound_event (portal/NSWS)
`evc`	P1	evc_request, expert, availability, coi, slot, assignment, schedule_run
`doc`	P2	document, document_version, fact, ocr_run
`cmp`	P2	approval_letter, obligation, evidence, verification
`dl`	P2	deadline, calendar_version, holiday, alert, ack, ladder, sim_clock
`plat`	P3	user_ref, role_scope, domain_event, ledger_entry, checkpoint, case_note, personal_data, outbox, delivery, idempotency_key, export
`an`	P3	projection tables, report_snapshot, scorecard_run

9.2 Every §17.1 module has one owner

Module	Owner
Institution portal	P1 backend · P3 UI
Workflow engine	P1
Case + deficiency	P1
Committee workspace	P1 backend · P3 UI
EVC queue + scheduling	P1
Pendency + SLA	P3 (projection) from P1 + P2 events
Obligation tracker	P2
Document lifecycle	P2
Verification	P2
Notifications + escalation	P2 (generation, escalation) · P3 (delivery)
Dashboards / reports	P3

Module	Owner
Identity + permissions	P3
Audit + provenance	P3
Integration	P1 (portal, NSWS) · P2 (DigiLocker)

9.3 Every §27 API has one owner

API	Owner
`POST /applications`, `/applications/{id}/submit`, `/cases/{id}/actions`	P1
`GET /cases/{id}/timeline`	P3
`GET /deadlines`, `/deadlines/{id}/explain`, `POST /deadlines/{id}/extensions`	P2
`GET /obligations`, `POST /obligations/{id}/evidence`, `POST /evidence/{id}/verify`	P2
`POST /documents`, `GET /documents/{id}/facts`, `POST /facts/{id}/confirm`	P2
`GET /evc/queue`, `POST /evc/schedule:propose`, `:publish`	P1
`GET /alerts`, `POST /alerts/{id}/ack`	P2
`GET /dashboards/pendency`, `GET /reports/{type}`	P3
`GET /ledger/verify`, `POST /exports`	P3
`POST /admin/rulesets`, `/validate`, `/approve`	P1
`POST /sim/clock`	P2

9.4 Every §28.1 tier item has one owner

Tier	Item	Owner
1	Thin institution portal	P3 UI, P1 backend
1	Versioned routes ≥5 categories + static validation	P1 (all 21 + new institution)
1	State machine, guards, reverts with reasons	P1
1	Deficiency loop	P1
1	EVC queue	P1
1	Deadline engine (calendars, working days, pauses, extensions, explanations)	P2
1	Reminders, escalation ladder, acks, dedup	P2 (delivery: P3)
1	Obligation register (EoA 6-month)	P2
1	Lease rule with manual facts	P2
1	Role dashboards	P3
1	Hash-chained ledger + verify	P3
1	Keycloak roles	P3
1	Synthetic generator	P1 (cases), P2 (documents)
1	Scorecard	P3
1	Time-travel clock	P2 (UI: P3)
2	OCR Stage 0 + handwriting detection + lease extraction with commit-before-reveal	P2 (UI: P3)
2	CP-SAT EVC dates with explanations	P1
2	Rule-based early warning	P2

Tier	Item	Owner
2	Rule-change impact replay	P1
2	Cited case summaries (local LLM)	P1 (optional)
2	s.63-ready export bundle	P3
2	BPMN view of route versions	P1
2	Simulated NSWS/AICTE adapter	P1
3	Handwritten recognition (Stages 3–4)	P2 (roadmap only)
3	Merkle proofs + external witness	P3 (roadmap)
3	Cross-document consistency	P2 (roadmap)
3	APH helper with citations	P1 (roadmap)
3	Real adapters (NSWS, DigiLocker, SMS)	P1 / P2 / P3 respectively (roadmap)
3	Predictive model on real logs; pm4py study	P3 (roadmap; pm4py is AGPL, so keep it outside the core)
3	Hindi UI, GIGW/STQC audit	P3 (roadmap)

9.5 Cross-cutting items

Item	Owner
Licence hygiene (no Camunda 8, immudb, n8n, AGPL in core; pypdfium2 not PyMuPDF)	P3 runs the CI licence check; each owner keeps their own dependencies clean
Indic OCR model licence (submission gate)	P2
Operaton spike	P1
Threat model (§26)	P3 leads; each owner fixes findings in their module
Demo script and rehearsal	All (section 12)
README, architecture slide, final presentation	P3 assembles; each writes their own section

---

10. Week-by-week plan

Checkpoints come from §28.3: W1 events end to end · W2 scorecard green · W4 Tier 2 behind flags · W6 freeze and rehearsal.

Week	P1	P2	P3
0 (Day 0)	Contracts (section 7)	Contracts	Contracts + repo/CI/compose
1	Registry + r1 routes (all 21) + state machine; submit returns route	Deadline engine + calendars + clock; golden date tests	UoW, ledger append + verify, Keycloak/Cerbos, institution and officer screen shells
2	Deficiency loop + committee backend; case generator + answer key	Timer worker, alerts, ladder, acks; obligation register + EoA 6M; letters	Outbox + notifier; replay harness; scorecard v1; **W2: scorecard green on Tier 1**
3	EVC queue, expert pool, COI, selector stub; EDF baseline	Doc upload + ClamAV + router + Tesseract; Stage 0 OCR fixes; digit normalisation + date grammar	Dashboards (pendency, workload, obligations); alert inbox; "Why is this due?" panel

Week	P1	P2	P3
4	CP-SAT scheduler + explanations; rule-change impact replay; r3-DEMO switch	Indic model wrapper; handwriting detection (Stage 2); lease stop-AY; OCR eval set; **model-licence check**	OCR review screen (commit-before-reveal); EVC schedule view; checkpoints; **W4: Tier 2 behind flags**
5	BPMN view; NSWS/portal mocks; Operaton spike write-up; hardening	Early warning; DigiLocker mock; seeded-reviewer-error metric; restart catch-up test	Threat-model tests; cross-role matrix; s.63 export; accessibility pass
6	Bug fixes; demo steps 1–4, 10	Bug fixes; demo steps 5–9	Bug fixes; demo steps 11–13; **W6: freeze + rehearsal**

48-hour fallback (only if starting cold). The same ownership on a compressed timeline:

- **h0–6:** everyone does Day 0.
- **h6–24:** P1 engine; P2 deadlines; P3 ledger + auth.
- **h24–36:** P1 deficiency + EVC queue; P2 alerts + obligations; P3 UIs + dashboards.
- **h36–44:** P1 generator; P2 time travel; P3 scorecard.
- **h44–48:** rehearsal.
- OCR and CP-SAT drop to Tier 2.

---

11. Effort balance and rebalancing levers

These are rough estimates, not measured effort, like the ones in the split you were given.

- **P1** is a medium load. The state machine is conceptually simple, but all 21 routes plus CP-SAT are real work.
- **P2** is the heaviest load. OCR and the deadline engine are both exacting. Moving delivery, notifier and the review UI to P3 already lightens it.
- **P3** is a medium-to-heavy load, because the frontend is large.

If someone falls behind, move whole items. Never share a file.

If behind	Move this item	To	Why it is safe
P2	OCR evaluation set + document generator	P1	Separate folder (`generator/documents/`), consumes only the `fact` format
P2	Early warning (Tier 2)	P3	Reads deadlines through a view; no writes
P3	EVC schedule view + committee screens	P1	Self-contained pages under a route P3 assigns
P3	s.63 export bundle	P1	Reads the ledger only
P1	BPMN view, cited summaries	Drop	Tier 2 optional

When an item moves, update `docs/OWNERSHIP.md` in the same integration so ownership stays single.

---

12. Demo: who presents what

The demo script is §29. Each step has one owner, who also owns its e2e test file.

Step	What happens	Owner
1	Sunrise Institute submits; route from `aph-2024-27@r1` with citation	P1
2	2 deficiencies; AICTE clock pauses, institution clock starts	P1 (clocks shown by P2)
3	Re-Scrutiny on open items only; advance to EVC queue	P1
4	CP-SAT dates for 40 EVCs, 0 conflicts vs EDF; panels from the stand-in selector	P1
5	Time travel +20 days; T-3 reminder; escalation RO → Bureau; one alert per level	P2
6	EoA with deficiencies creates 3 obligations, due in 6 months	P2
7	Kannada lease deed: router, digit normalisation, type-then-reveal, stop-AY, 18-month reminder	P2 (screen by P3)
8	Handwritten date detected and routed to manual entry	P2
9	30-year extension verified; obligation MET	P2
10	"Why is this due?" then the r1 vs r3-DEMO switch	P1 (panel by P3)
11	Tamper test: `/ledger/verify` reports the break	P3
12	Scorecard on the 600-application cycle, incl. the notifier outage	P3
13	Dashboards update: pendency drops, workload rebalances	P3

Questions from evaluators (§31.1) go to the owner of the area: rules and EVC to P1; OCR, deadlines and law-based dates to P2; security, privacy, audit and data to P3.

---

13. What is already built (Week 1 slice)

The starter project is already in the repository, so nobody starts from an empty folder. It proves the three parts are genuinely connected: **an institution submits an application in the UI → Person 1's workflow picks the route → Person 2's engine opens a clock → Person 3's ledger records it and the UI shows the state and audit timeline.**

Verified on 24 Oct 2026:

- **Tests:** 31 automated tests pass (unit + end-to-end).
- **Docker:** the full flow was run through the real API in Docker.
- **UI:** the UI was checked against it in a browser.

13.1 Person 1: already in place

Done	Where
Ruleset registry: loads, validates and pins `aph-2024-27@r1`; refuses `r3-DEMO` outside demo mode	`backend/app/workflow/registry.py`
All 21 Table 2.3 routes + NEW_INSTITUTION, with labels; static check that SHC appears exactly where EVC does	`rulesets/aph-2024-27/r1/routes.yaml`
`clocks.on_submit` in YAML decides which clocks a submission opens (no hard-coding)	`routes.yaml`
Institutions, applications, stage visits; create + submit in one transaction	`backend/app/intake/`
APIs: `GET /institutions`, `POST /applications`, `POST /applications/{id}/submit`, `GET /applications`, `GET /rulesets`, `GET /rulesets/{key}`	`intake/api.py`, `workflow/api.py`
3 fictional DEMO institutions	`generator/cases/seed_demo.py`

Done	Where
Tests: all routes, corrected rows, static validation, overlay refusal	`backend/tests/test_rulesets.py`

Your next steps:

1. The state machine and transitions from `routes.yaml`, starting with `record_deficiency` and `submit_compliance`. They call `DeadlinePort.pause_internal / resume_internal`, which already exist.
2. The deficiency loop with items.
3. The committee backend.
4. The EVC queue.
5. The scale generator and answer key.

13.2 Person 2: already in place

Done	Where
Deadline rule parser with validation (clock kind/owner, source status, duration XOR computed)	`backend/app/deadlines/rules.py`
Due-date math: Indian calendar dates, calendar days, months with month-end clamp	`rules.py`
Deadline engine: open / close / pause / resume; LEGAL clocks refuse to pause; one live clock per subject+rule	`backend/app/deadlines/service.py`
Explanations on every deadline ("why is this due?"), labelling demo assumptions	`service.py`
System clock with demo-only time travel (`GET/POST /sim/clock`)	`backend/app/deadlines/clock.py`
APIs: `GET /deadlines`, `GET /deadlines/{id}/explain`	`deadlines/api.py`
r1 rules incl. AICTE_PROCESSING (demo assumption), APPEAL_7D, DEPOSIT_15D, EOA_DEFICIENCY_6M, LEASE_EOA_STOP	`rulesets/aph-2024-27/r1/deadlines.yaml`
Tests: month-end clamp, IST dates, invalid rules	`backend/tests/test_deadline_rules.py`

Your next steps:

1. Working-day calendars (`calendars/`).
2. The timer worker.
3. Alerts with the escalation ladder and acks; enqueue to the outbox.
4. The lease stop-year computation.
5. Approval letters and obligations (`compliance/`).
6. The document pipeline and OCR (`documents/`).

13.3 Person 3: already in place

Done	Where
Unit of work (one transaction per command); port registry; module auto-discovery	`backend/app/platform/db.py`, `registry.py`, `backend/app/main.py`
Event store + hash-chained ledger; schema validation that forbids free text; `GET /ledger/verify`	`platform/ledger.py`, `platform/models.py`
Notes, personal data and outbox tables + ports	`platform/records.py`
Demo auth (X-Demo headers) + policy with institution-only-own-records rule; `GET /cases/{id}/timeline`	`platform/auth.py`, `platform/api.py`

Done	Where
React UI: role switcher, create + submit, route, clocks with "why", timeline, ledger check	`frontend/src/`
Docker Compose (db + backend + frontend), CI (tests, contracts, migration heads, three-way contract approval)	`docker-compose.yml`, `.github/`
Contract and migration checks	`scripts/check_contracts.py`, `scripts/check_migration_heads.py`
Tests: tamper and deletion detection, schema rejection, models match migrations	`backend/tests/test_ledger.py`, `test_schema_matches_models.py`

Your next steps:

1. Keycloak + Cerbos to replace demo auth.
2. The Idempotency-Key middleware.
3. The notifier that reads the outbox, with retries and delivery logs.
4. Signed independent checkpoints.
5. The replay harness.
6. Dashboards.

---

14. Branches and integration

```
main             always runs; changed only by integration
 ├── person-1   Person 1 works here
 ├── person-2   Person 2 works here
 └── person-3   Person 3 works here
```

Every day:

```
git switch person-2                   # your own branch
# ...code...
python -m pytest                      # must pass before you push
git add -A
git commit -m "Add escalation ladder"
git push
```

Every 2–3 days: integration.

1. All three person branches are merged into a short-lived `integration/YYYY-MM-DD` branch, in this order:
    - `contracts/`;
    - Person 3's platform;
    - Persons 1 and 2.
2. Conflicts are resolved and every check runs: tests, the contract check, migration heads, and a frontend build.
3. Anything a person changed **outside their own folders** is flagged and discussed before merging.
4. A pull request goes to `main`. Another team member approves it, and it merges.

Right after each integration, everyone syncs:

```
git switch person-1
git pull origin main
git push
```

Why integrate every 2–3 days and not only at the end:

- Two branches can merge without a conflict and still break the app, for example a function called in one shape and written in another.
- Small integrations catch this early, while the change is still fresh.
- Months of divergence surface only in demo week.

Commit messages: short, imperative, about the change (`Add deficiency loop transitions`, `Fix month-end clamp`).

Repository settings (repo owner, once): under Settings → Branches, add a rule for `main`:

- Require a pull request with 1 approval and Code Owner review.
- Require the `backend`, `frontend` and `all-three-approve` checks.
- Disallow bypassing these settings.

Private repositories need GitHub Pro for this; GitHub Education gives students Pro for free.